LexiScan — AI Legal Document Analyzer

Full-Stack Enterprise Architecture & Technical Specification Document

Project Title:	LexiScan (Legal Document Intelligence)	**Curriculum / Sem:**	JFSD — 5th Semester Capstone
Author / Dev:	Full Stack Engineering Team	**Architecture:**	Multi-Tier Containerized Micro-Services
Deployment:	Docker Compose (Multi-Container)	**Documentation Date:**	October 2026 | Version 1.0

1. Executive Summary & Purpose

LexiScan is an intelligent, full-stack legal technology application designed to analyze complex contracts (such as rental leases, non-disclosure agreements, master services agreements, and employment contracts) in PDF and Microsoft Word (DOCX) formats. It parses unstructured legal prose, normalizes text, segments numbered and headed clauses, identifies named legal entities (parties, financial amounts, durations, deadlines, and governing jurisdictions), flags risky or one-sided obligations, and scores the contract on a standardized 0–100 risk scale. Additionally, it compiles and exports an official executive audit report in PDF format.

The system aims to democratize legal comprehension for individuals, tenants, freelancers, and small business owners, providing plain-language clarity before executing binding agreements while adhering to strict compliance disclaimers.

2. Comprehensive Technology Stack

Layer / Tier	Technology	Version	Key Role & Responsibility
Frontend UI	React.js (SPA)	18.3.1	Declarative component UI, document upload, real-time polling
Frontend Build	Vite	8.3.5	Ultra-fast ESM bundling, asset optimization, production build
Frontend Styling	Modern CSS3 (Vars)	Native	Calm & Cold design system, dual Light/Dark mode themes
HTTP Client	Axios	1.7.7	REST API client with JWT Authorization bearer interceptors
Backend Framework	Spring Boot	3.4.3	Enterprise Java REST API, Spring MVC, Spring Data JPA
Programming Runtime	Java SE / Temurin	JDK 17 LTS	Modern record classes, pattern matching, parallel streams
Security & Auth	Spring Security + JJWT	0.12.6	BCrypt encryption, stateless JWT authentication, RBAC, CORS
Database Engine	PostgreSQL	16 Alpine	Relational ACID storage for users, documents, clauses, rules
DB Migrations	Flyway	10.x	Automated database versioning, schema creation & rule seeding
PDF Parsing	Apache PDFBox	3.0.4	Text extraction, font parsing, document structure stripper
DOCX Parsing	Apache POI (ooxml)	5.4.0	Word XML paragraph traversal and heading decomposition
PDF Report Engine	OpenPDF	2.0.3	Dynamic generation of official audit reports with legal summary
API Documentation	SpringDoc OpenAPI	2.8.5	Interactive Swagger UI at /swagger-ui.html for API testing
Web Server / Proxy	Nginx Alpine	1.31.8	Static asset web server hosting compiled React SPA
Containerization	Docker & Compose	v2.x	Multi-container microservices, healthchecks, network isolation

3. System Architecture & Component Interaction

LexiScan follows a decoupled, containerized three-tier architecture ensuring high maintainability, fault isolation, and clean separation of concerns.

Tier	Component	Port & Protocol	Responsibility & Data Flow
Presentation Tier	React SPA + Nginx	Port 5173 (HTTP)	Renders user interface, dark mode theme engine, dropzone file upload, polling mechanism for real-time status updates.
Application Tier	Spring Boot 3 API	Port 8081 (Internal 8080)	Dispatches REST requests, performs JWT verification, validates file magic bytes, triggers asynchronous NLP execution thread pool.
NLP Processing	Java NLP Engine	In-Process Async Thread Pool	Extracts text via PDFBox/POI, segments clauses, extracts regex entities, executes heuristic risk rules matching, calculates score.
Data Storage Tier	PostgreSQL 16 DB	Port 5432 (Internal)	Stores relational tables (Users, Documents, Clauses, Analysis Results, Risk Rules) with cascading foreign key integrity.
Document Storage	Private File System	Volume: /app/uploads	Saves raw uploaded contracts using cryptographically secure UUID file names to eliminate directory traversal attacks.

4. End-to-End Document Analysis Workflow

- **Step 1: Document Upload & Magic Byte Verification:** The client uploads a contract through the React dropzone. The Spring Boot backend inspects the stream's first 4 bytes (magic bytes: %PDF for PDFs or PK.. for Word DOCX) to prevent malicious executable masquerading. Stored files are assigned random UUIDs in /app/uploads.
- **Step 2: Database Registration & Transaction Initialization:** A new record is created in documents table with status UPLOADED and tied to the authenticated user's UUID.
- **Step 3: Asynchronous Trigger:** The frontend calls POST /api/documents/{id}/analyze. The service updates status to PROCESSING and delegates the analysis task to a managed Spring ThreadPoolTaskExecutor.
- **Step 4: Text Extraction & Normalization:** For PDF files, PdfTextExtractor runs PDFTextStripper. For DOCX, DocxTextExtractor traverses all XML body paragraphs. The extracted string is normalized (whitespace collapsed, line-breaks harmonized).
- **Step 5: Clause Segmentation & Classification:** The NLP engine segments clauses using regex boundary detection (numbered clauses e.g. 1. TERM, 2. LIABILITY or uppercase headings). Each clause is classified into legal categories (e.g., PAYMENT, TERMINATION, INDEMNITY, CONFIDENTIALITY, GOVERNING LAW, DURATION) using weighted keyword dictionaries.
- **Step 6: Entity Extraction:** Specialized regex patterns scan for financial currencies and amounts (INR, USD, EUR, $), timeframes and durations (months, days, years), calendar dates, and counter-party identities.
- **Step 7: Risk Rule Matching & Scoring:** Each clause is matched against active database rules and contextual risk patterns (e.g. uncapped indemnity, unilateral short termination notices, excessive late payment interest >24%, non-refundable deposits). Each clause receives a risk score (0–100) and risk tier (LOW, MEDIUM, HIGH).

- **Step 8: Aggregate Scoring & Persistence:** The overall document risk score is calculated as the capped arithmetic mean: min(100, round(sum(clause scores) / clause count)). Records are written to analysis_results and clauses, and document status is updated to ANALYZED.
- **Step 9: Interactive UI Display & PDF Report Export:** The React client polls GET /api/documents/{id}, detects ANALYZED status, renders the circular score gauge, entity badges, plain-language summary, and searchable clause explorer, and enables on-demand OpenPDF report downloads.

5. Database Schema & Data Models

PostgreSQL maintains relational consistency with cascading foreign key deletions:

Table Name	Primary Key	Foreign Keys / Constraints	Core Columns & Attributes
users	id (UUID)	UNIQUE(email)	name, email, password_hash (BCrypt), role (USER/ADMIN), enabled, created_at
documents	id (UUID)	user_id -> users(id) ON DELETE CASCADE	file_name, stored_path, file_type (PDF/DOCX), file_size, status (UPLOADED/PROCESSING/ANALYZED/FAILED), uploaded_at, analyzed_at
analysis_results	id (UUID)	document_id -> documents(id) ON DELETE CASCADE	overall_risk_score (INT), overall_risk_level (VARCHAR), summary (TEXT), key_entities (JSON text), clause_count, high_risk_count, medium_risk_count
clauses	id (UUID)	document_id -> documents(id) ON DELETE CASCADE	clause_index (INT), clause_text (TEXT), category (VARCHAR), risk_level (LOW/MED/HIGH), risk_score (INT), risk_reason (VARCHAR)
risk_rules	id (UUID)	None (System Rules Engine)	category, pattern (regex), risk_level, risk_score, risk_reason, active (BOOLEAN)

6. REST API Endpoints Specification

HTTP Method	Endpoint Path	Auth Required	Description & Output
POST	`/api/auth/register`	Public	Registers a new user account; returns JWT bearer token and user profile.
POST	`/api/auth/login`	Public	Authenticates email & password via BCrypt; issues stateless JWT session token.
GET	`/api/auth/me`	Bearer JWT	Returns current authenticated user details and permission role (USER/ADMIN).
POST	`/api/documents`	Bearer JWT	Multipart upload for .pdf or .docx contracts; validates magic bytes & size.
GET	`/api/documents`	Bearer JWT	Paginated list of the current user's uploaded documents with statuses.
GET	`/api/documents/{id}`	Bearer JWT	Fetches status, file details, and risk level of a user's document.
DELETE	`/api/documents/{id}`	Bearer JWT	Deletes document record, associated clauses, analysis results, and disk file.
POST	`/api/documents/{id}/analyze`	Bearer JWT	Triggers asynchronous NLP clause analysis, risk pattern evaluation, and scoring.
GET	`/api/documents/{id}/analysis`	Bearer JWT	Returns overall score, summary, detected entities, and full clause breakdown.
GET	`/api/documents/{id}/clauses`	Bearer JWT	Retrieves clauses with optional category and riskLevel query filters.
GET	`/api/documents/{id}/report`	Bearer JWT	Dynamically compiles and streams downloadable OpenPDF audit report.
GET	`/api/admin/stats`	Role: ADMIN	Platform analytics: total users, total documents, and completed analyses count.
GET	`/api/admin/rules`	Role: ADMIN	Lists all active legal risk pattern rules in the heuristic database.
GET	`/actuator/health`	Public	Spring Boot Actuator operational health status ({"status":"UP"}).

7. Security, Privacy & Integrity Measures

- **Stateless Authentication:** Uses JSON Web Tokens (JJWT) with HMAC-SHA256 digital signatures, expiring in 120 minutes. Passwords use BCrypt hashing with standard salt rounds.
- **Binary Magic Byte Verification:** Validates file signatures at the stream header level to prevent malicious file uploads disguised with altered file extensions.
- **Anti-Path Traversal File Storage:** Stored contract files are decoupled from their original filenames and written to disk as randomized UUIDs.
- **Ownership Isolation:** Database queries enforce user ID filtering at the JPA repository layer; unauthorized requests to other users' documents return HTTP 404.

- **Cross-Origin Resource Sharing (CORS):** Spring Security explicitly handles preflight OPTIONS requests and binds allowed origin patterns for localhost:5173 and 127.0.0.1:5173 with full HTTP method support.

8. User Interface, Cold Palette & Dark Mode

The frontend is crafted around a **Nordic Frost & Glacial Slate** calm color palette with built-in instant dark mode switching:

- **Dual Theme Engine:** Supports Light (Frosted Ice `#f0f4f8`) and Dark (Midnight Glacier `#0b111e`) modes with instant CSS variable switching and persistent local storage.
- **Drag & Drop Dropzone:** Visual contract upload box with file size calculation, file preview, and upload progress spinners.
- **Risk Score Meter:** Color-coded circular score dial (0–100) with High/Medium/Low risk badge indicators and clause counter cards.
- **Real-Time Clause Search:** Client-side instant keyword filtering across contract text, category tags, and risk explanations.
- **Administrator Telemetry:** Specialized metrics card surfacing real-time system counts when signed in with admin credentials.

9. Operational Run & Deployment Guide

- **Prerequisites:** Docker Desktop running on the host system.
- **Directory:** Navigate to the project root: cd "D:\JFSD-5th sem"
- **Launch Services:** Run docker compose up --build -d to build containers, run migrations, and start Nginx, Spring Boot, and PostgreSQL.
- **Access Application:** Open http://localhost:5173 in any browser.
- **Access API Documentation:** Open http://localhost:8081/swagger-ui.html for live OpenAPI Swagger documentation.
- **Stop Services:** Run docker compose down to stop and tear down containers while keeping database data intact.